SecondThought: Confidence-Gated Reasoning for Tool Calls in Small Local Models

Ali Ashter
hello@ashter.dev

October 2026

Abstract

Letting a model think before it calls a tool improves the call, but costs six to seven times the output tokens and, we find, makes models call tools they should leave alone. We propose gating: make the normal no-think call, read its mean token log-probability, and rerun with thinking only when that confidence is low. On the Berkeley Function Calling Leaderboard (1,398 single-turn items including 240 where no tool fits) with Qwen3-1.7B, 4B and 8B, the call's own mean log-probability predicts whether it is correct with AUROC 0.82 to 0.84, and the probability of the first token being a tool call predicts whether a tool is needed with AUROC 0.96 to 0.98, better than asking the model directly. Thinking on the least confident 30% of requests recovers 66% and 70% of the accuracy gain of always thinking at 1.7B and 4B, and exceeds always thinking at 8B (0.877 against 0.874), at about half the output tokens, because it avoids the thinking that talks models into wrong calls. Confidence ranking beats random selection at every budget. The gate needs no training and no extra model; it reads numbers the server already returns. We also report twelve single-factor interventions on the same data. Code and every response: `https://github.com/alleyxd/SecondThought`.

1 Introduction

Small open models now handle single tool calls well enough to run local and on-device agents. Reasoning modes make them better still: on our data, thinking before the call adds 3 to 5 points of accuracy on requests that need a tool. But it multiplies output tokens by six to seven and adds seconds of latency to every request, and it has a side effect we did not expect: on requests where no tool fits, thinking lowers the rate of correctly declining by about 6 points. The model reasons its way into a call.

Most requests do not need the extra reasoning; the direct call is already right. The question is which ones do. Our earlier work showed that a model's next-token probabilities, read in a single forward pass, are informative about its decisions once calibrated [1]. Here we use the same idea for routing: the log-probabilities of the direct tool call tell us how sure the model was, and we spend thinking only where it was not.

Contributions:

- Two free signals from the direct call: its mean token log-probability predicts correctness, and the probability of the first token being a tool call predicts whether a tool is needed. Both beat asking the model.
- A training-free gate that thinks only on low-confidence calls, evaluated across budgets against random gating, always thinking and an oracle.
- Twelve single-factor interventions on the same data (descriptions, names, distractors, retrieval, forced calls, schema repair) as context for where tool calls fail.

2 Related work

BFCL [3], which grew out of Gorilla [4], is the standard function-calling benchmark; we use its single-turn categories and matching rules. Wu et al. treat whether to call a tool as a decision problem [5]; we give a concrete, cheap estimator of that decision. Tool choice can be read and steered from hidden states [6]; we read only output log-probabilities, which any OpenAI-compatible server returns. Retrieval for small agents [7, 8] and interface design for recovery [9] are complementary. Gating expensive computation on confidence is an old idea in model cascades; our contribution is showing which signals work for tool calls and that gating improves on always thinking rather than only approximating it. Our previous studies on single-pass decisions [1] and structured output [2] motivate the method and setup.

3 Setup

Data. BFCL v3: simple (400), multiple (200), live_simple (258), live_multiple (300 sampled with a fixed seed) and irrelevance (240, where the correct behaviour is not to call). 1,398 items.

Scoring. A BFCL-style AST check: the expected function, every argument among the accepted values, re-

	mean_lp	min_lp	p_call	p_probe
Is the direct call correct? (tool items)				
1.7B	0.84	0.83	0.52	0.66
4B	0.82	0.82	0.53	0.80
8B	0.82	0.82	0.68	0.77
Is a tool needed? (all items)				
1.7B	0.94	0.93	0.97	0.82
4B	0.94	0.92	0.98	0.96
8B	0.94	0.92	0.96	0.94

Table 1: AUROC of each signal.

quired arguments present, no unknown arguments. Irrelevance items score 1 for no call. We report accuracy over all 1,398 items, so declining correctly counts.

Models. Qwen3-1.7B, 4B and 8B [10] in bf16 on vLLM 0.29.0 [11] with the Hermes tool parser, greedy decoding.

Signals. From the direct (no-think) call with token log-probabilities: *mean_lp* and *min_lp*, the mean and minimum log-probability over the generated tokens, and *p_call*, the probability that the first token is `<tool_call>`. For comparison, *p_probe*: a separate one-token question asking whether any available tool fits (A/B), normalised over the two answers.

Gate. Make the direct call. If it is a tool call and its mean_lp is among the lowest b fraction, rerun with thinking and use that answer; otherwise keep the direct answer. Requests the direct call declined are never sent to thinking. Cost is the output tokens of both passes.

4 Results

4.1 The call's own log-probabilities are the best signals

The mean token log-probability of the direct call separates right from wrong calls with AUROC 0.82 to 0.84 at every size (Table 1). The first-token probability of a tool call is the best signal for whether a tool is needed at all, 0.96 to 0.98, and it beats explicitly asking the model, which is weakest at 1.7B (0.82). Both come from the call the agent makes anyway, so they cost nothing extra.

4.2 Gating keeps most of the gain at half the cost

Table 2 shows accuracy as the thinking budget grows. At 30%, the gate reaches 0.824, 0.873 and 0.877, recovering 66% of the gain of always thinking at 1.7B and 70% at 4B, and slightly exceeding always thinking at 8B. It uses 161 to 188 output tokens per request against 317 to 364 for always thinking. Random selection of the same number of requests does worse at every budget (0.818, 0.857 and 0.870 at 30%), so the ranking carries the gain.

	1.7B	4B	8B
direct (no thinking)	0.803	0.850	0.858
always think	0.835	0.883	0.874
gate, 10% budget	0.813	0.863	0.869
gate, 20%	0.819	0.871	0.871
gate, 30%	0.824	0.873	**0.877**
gate, 40%	0.825	0.873	0.876
random, 30%	0.818	0.857	0.870
oracle	0.890	0.921	0.905
tokens: always think	317	354	364
tokens: gate 30%	161	179	188

Table 2: Accuracy over all 1,398 items. Budget is the share of requests sent to thinking. Oracle thinks exactly when thinking is right and direct is wrong. Tokens are mean output tokens per request; the gate counts both passes.

Beyond 30% the curve is flat: the remaining requests are mostly already correct.

Requests that are not sent to thinking are exactly the direct answers, and a few of the ones that are can get worse, since thinking occasionally turns a correct call into a wrong one. Its value is choosing where to spend reasoning, not improving every response. The 8B result is the exception that shows why that matters: always thinking makes some declines into wrong calls, and the gate avoids those, so it beats thinking on everything.

4.3 Thinking hurts abstention

On the 240 irrelevance items, thinking lowers the rate of correctly declining from 0.863 to 0.800 at 1.7B, 0.879 to 0.817 at 4B and 0.858 to 0.800 at 8B. Never sending declined requests to thinking protects these, which is a simple rule worth applying to any gating scheme.

4.4 Context: single-factor interventions

We also ran twelve conditions on the same items with thinking off, scoring the 1,158 tool-needing items (Table 3). Removing parameter descriptions costs 16 to 18 points, renaming functions about one. Distractor tools hurt small models most. BM25 retrieval kept the right tool in its top five for 88% of items and scored below sending all 32 distractors at 4B and 8B; an embedding retriever kept it 97% of the time and beat sending everything. Forcing a call adds up to 4 points but declines nothing. Validating calls against the schema and retrying triggered on about 1% of items: most failures were schema-valid calls with the wrong function or values (180, 132 and 132 base failures at 1.7B, 4B and 8B) or text answers with no call (58, 42 and 19), which is what the gate's thinking pass fixes.

Condition	1.7B	4B	8B
base	0.789	0.845	0.861
think	0.842	0.896	0.889
no function descriptions	0.764	0.824	0.864
no descriptions at all	0.612	0.680	0.697
names `tool_k`	0.798	0.839	0.852
+8 distractors	0.715	0.827	0.839
+32 distractors	0.650	0.789	0.819
+32, BM25 top 5	0.693	0.742	0.746
+32, BM25 top 10	0.699	0.771	0.785
+32, embedding top 5	0.760	0.814	0.828
`tool_choice=required`	0.826	0.867	0.866
validate and retry	0.791	0.849	0.864

Table 3: Single-factor interventions, accuracy on the 1,158 tool-needing items.

5 Using it

- Request log-probabilities on the normal tool call; most servers return them.
- If the call looks unsure (low mean log-probability), retry with thinking. Calibrate the cut-off on a few dozen labelled requests, or start by thinking on the least confident quarter.
- Never send a declined request to thinking.
- Use the first-token probability of a tool call, not a separate yes/no question, to monitor abstention.

6 Limitations

One model family, greedy decoding and single-turn BFCL categories; on this data thinking adds only 2 to 3 points overall, which bounds what any gate can gain. Multi-turn agents, where a wrong call compounds, may benefit more but are untested. Budgets are fixed fractions of the evaluation set; in deployment a threshold must be fixed in advance, which we approximate with cross-validation in the code. Latencies were measured under batched load and are not reported.

7 Conclusion

A tool call's own log-probabilities say when it is likely wrong and when no tool was wanted. Spending reasoning only on unsure calls keeps most of the benefit of thinking, at half the tokens, and avoids its habit of talking models into calls they should not make.

References

[1] A. Ashter. Decisions in one forward pass: typed answers from local language models, and making their yes/no probabilities mean something. Zenodo, 2026. doi:10.5281/zenodo.23555119.

[2] A. Ashter. The schema tax is a schema design tax: when constrained JSON decoding costs accuracy, and when it helps. `https://github.com/alleyxd/SchemaTax`, 2026.

[3] S. G. Patil, H. Mao, F. Yan, C. Ji, V. Suresh, I. Stoica, J. E. Gonzalez. The Berkeley Function Calling Leaderboard (BFCL): from tool use to agentic evaluation of large language models. ICML 2025.

[4] S. G. Patil, T. Zhang, X. Wang, J. E. Gonzalez. Gorilla: large language model connected with massive APIs. arXiv:2305.15334, 2023.

[5] Q. Wu, S. Das, M. Amani, et al. To call or not to call: a framework to assess and optimize LLM tool calling. arXiv:2605.00737, 2026.

[6] Z. Wu, Z. Wang, S. Cho, et al. Tool calling is linearly readable and steerable in language models. arXiv:2605.07990, 2026.

[7] B. Patel, D. Belli, A. Jalalirad, et al. Dynamic tool dependency retrieval for lightweight function calling. arXiv:2512.17052, 2025.

[8] S. Huang, M. Zhang, B. Hu, et al. ToolOmni: enabling open-world tool use via agentic learning with proactive retrieval and grounded execution. arXiv:2604.13787, 2026.

[9] Z. Wang. Callability is not operability: controlled interface interventions for LLM agents. arXiv:2608.23628, 2026.

[10] A. Yang, A. Li, B. Yang, et al. Qwen3 technical report. arXiv:2505.09388, 2025.

[11] W. Kwon, Z. Li, S. Zhuang, et al. Efficient memory management for large language model serving with PagedAttention. SOSP 2023. arXiv:2309.06180.